Chance-Constrained Communication Planning for Stochastic Multi-Agent Projects

Jaye Heglund[1], Mikihisa Yuasa[1], Hamid Osooli[1], and Huy T. Tran[1]

***Abstract*— TODO: this framing kinda sucks lol**

Multi-agent reinforcement learning (MARL) is a promising approach to designing agents that can solve complex, cooperative tasks. However, complete communications are not guaranteed in the real-world. Therefore, an important problem in MARL is in in minimizing communication cost while still satisfying a task's reliability specification. The key idea is that different parts of a task may require different levels of communication, and these differences should be considered when designing a team's joint policy. This paper implements this idea through a hierarchical model, called a coordination machine, that decomposes a cooperative task into subtasks. Policies are then optimized for choosing subtasks, allocating communication capacity to subtasks, and completing subtasks given their allocation. This paper uses the coordination machine to ensure that the optimized hierarchical policy satisfies a probabilistic task-completion specification while minimizing communication. This paper demonstrates the approach in two environments and achieves a significant reduction in communication cost compared to a fixed-communication baseline. A multi-robot demonstration of the method is also included.

I. Introduction

Multi-agent reinforcement learning is a promising approach for optimizing policies for teams of agents to solve complex tasks, like robot soccer [1] and traffic signal control [2]. However, autonomous agents typically act based on incomplete observations, which makes communication an important mechanism for coordinating their actions, particularly in cooperative settings. This challenge has led to a line of research studying MARL with communication (Comm-MARL), where common questions include when to communicate, what to communicate, and with whom to communicate [3]. One unstudied question though is what communication capacity (e.g., bandwidth, number of edges in a communication graph, etc.) is needed to ensure a complex task specification is satisfied (e.g., safely reaching a sequence of states with some probability). This question is important when deploying real-world multi-agent systems (MAS) in safety-critical applications, where one must ensure the design of agent policies and their required communication network is sufficient to satisfy a given task specification.

TODO: this framing suuuuucks, these are not actual approaches to comms allocation, this is kind of like a strawman. You want to set up stronger baseline comms allocation methods than this

Two relevant classes of Comm-MARL methods include those that learn policies where the communication capacity is fixed throughout a task (e.g., [4]–[7]), and those that allow the assumed communication capacity to change as a function of state or observation (e.g., [8]–[10]). These classes exist on opposite ends of a spectrum with respect to the problem considered here. Due to their fixed communication capacity, methods in the first class are practical with respect to real-world implementation, but may result in over-allocation of communication capacity, or may struggle to satisfy the task specification due to insufficient capacity. Meanwhile, methods in the second class may result in communication requirements that are impractical to implement, as they may change at every state.

TODO: this framing sucksssss, just be like "we allow flexible comms allocation"

Using the terminology of project planning, [11], a project consists of tasks (i.e., jobs, subtasks, activities) that must be successfully completed to complete the overall project. Projects are typically subject to precedence constraints that define (partial) task ordering and resource constraints within and across tasks.

TODO: be like, communication constraints haven't really been explored in project scheduling, which is an important thing to consider in distributed cases or whatever. (VERIFY THIS CLAIM!)

TODO: this framing also sucksss b/c there are plenty of papers that allocate comms in interesting ways. You don't want to This work proposes a method that lies between these two classes. The proposed approach uses a task decomposition and designs an algorithm to optimally allocate communication capacity to each subtask while ensuring a project-level

with respect to a given task specification. This hierarchical decomposition allows communication capacity to be allocated dynamically to reduce the over-allocation of communication resources while maintaining a communication-network design that is more practical for real-world implementation than prior methods (e.g., by allowing capacity to change across subtasks rather than at every state). The hierarchical approach also supports non-Markovian tasks, such as those requiring completion of sequential subtasks with safety constraints, in addition to typical Markovian tasks, trading off global optimality to prioritize practical implementation benefits and sample efficiency.

The proposed method uses a hierarchical model, called a coordination machine (CM), that decomposes a project completion specification into an equivalent reachabillity specification on a probabilistic automaton with edges defined by tasks. This work then formulates a mixed-integer bilinear

[1] Department of Aerospace Engineering, University of Illinois Urbana-Champaign, Urbana, IL 61801, USA {jheglun2, myuasa2, hosooli2, huytran1} @illinois.edu

program to optimize high-level policies that choose the subtasks to execute and allocate communication capacities to each subtask, minimizing inter-agent communication while satisfying a probabilistic task-completion constraint. The approach provides a median reduction in communication cost of 46% across potential task-completion requirements compared to baselines in two multi-agent tasks.

II. Related Work

One class of Comm-MARL methods learns a communication policy to send messages between agents given a fixed communication capacity. For example, SchedNet [4] and IMAC [5] optimize a message scheduler and message policy based on a global bandwidth constraint. DCG [6] optimizes decision policies assuming a fixed coordination graph that defines inter-agent communication. CASEC [7] learns a dynamic coordination graph topology, but assumes a fixed number of edges in the graph throughout the task. While these methods show promising results, their assumption of a fixed communication capacity, rather than capacity that can vary based on context, may result in over-allocation of communication resources. Additionally, they do not formally consider complex task specifications, such as those requiring non-Markovian rewards or safety constraints.

Another class of methods focuses on end-to-end learning of agent communication as a function of state or observation. IC3Net [8] uses a gating mechanism to learn when to communicate between agents, while [12], [13] use variations of a graph attention network to learn contextual communication as a function of the joint state. While the dynamic communication capacities offered by these methods reduces the actual number of messages sent, the lack of context provided for when various capacities are needed makes real-world implementations of such methods difficult and costly, since resources would have to be allocated to the provide the maximum capacity needed at all times. Please see [3], [14] for in-depth reviews of Comm-MARL methods.

Various hierarchical methods have also been proposed for MARL, including those exploring subteams [15], [16] and task decompositions [17], but these do not explicitly consider communication between agents. [18] considers subteams of a coordination graph, but does not explore the impact of resulting communications on task decompositions.

III. Preliminaries

A. Multi-Agent Reinforcement Learning

This paper models the low-level environment as a decentralized partially observable Markov decision process (Dec-POMDP) specified by the tuple $\mathcal{M} = (\mathcal{N}, \mathcal{S}, \{\mathcal{A}^i\}_{i\in\mathcal{N}}, P, \{\mathcal{O}^i\}_{i\in\mathcal{N}}, O, R, b_0, T, \gamma)$, where $\mathcal{N} = \{1, \ldots, n\}$ is the set of agents, $\mathcal{S}$ is the state set, $\mathcal{A} = \times_{i\in\mathcal{N}}\mathcal{A}^i$ is the joint action set where $\mathcal{A}^i$ is agent i's action set, $P : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the state transition function, $\mathcal{O} = \times_{i\in\mathcal{N}}\mathcal{O}^i$ is the joint observation set where $\mathcal{O}^i$ is agent i's observation set, $O : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{O})$ is the joint observation function, $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the reward function, $b_0 \in \Delta(\mathcal{S})$ is the initial state distribution, T is the time horizon, and $\gamma \in (0, 1]$ is the discount factor.

At time step t, the agents execute a joint action $a_t = (a_t^1, \ldots, a_t^n)$, after which the system transitions from state s_t to state $s_{t+1} \sim P(s_t, a_t)$, each agent receives a local observation o_t^i based on the joint observation $o_t = (o_t^1, \ldots, o_t^n) \sim O(s_t, a_t)$, and the team receives reward $r_t = R(s_t, a_t, s_{t+1})$. Each agent tracks its own action-observation history $h_t^i = (o_0^i, a_0^i, \ldots, o_{t-1}^i, a_{t-1}^i, o_t^i) \in \mathcal{H}^i = (\mathcal{O}^i \times \mathcal{A}^i)^* \times \mathcal{O}^i$, which it uses to select actions according to its policy $\pi^i : \mathcal{H}^i \to \Delta(\mathcal{A}^i)$. The joint policy is then $\pi = (\pi^1, \ldots, \pi^n)$. A common goal in MARL is to learn an optimal joint policy, π^*, which maximizes the expected sum of discounted rewards, $\mathbb{E}[\sum_{t=0}^{T-1} \gamma^t r_t]$.

IV. Method

TODO: formalize the "full" problem of choosing which tasks to execute to complete a given project. The hierarchical policy is an approximation to the "full" solution to that problem, so make it clear that you're accepting that tradeoff.

TODO: Check the project scheduling literature to see if they already have stuff on this. I'm not seeing anything

The high-level action is a pair $(i, c) \in \mathcal{I} \times \mathcal{C}$, where i selects a subtask and c specifies its communication capacity. We use a factored high-level policy $\tilde{\pi} = (\tilde{\pi}_e, \tilde{\pi}_c)$, whose joint distribution is

$$\tilde{\pi}(i, c \mid \tilde{s}) = \tilde{\pi}_e(i \mid \tilde{s}) \mathbb{1}\{c = \tilde{\pi}_c(i)\}. \tag{1}$$

Here, $\tilde{\pi}_e : \tilde{\mathcal{S}} \to \Delta(\mathcal{I})$ is a stochastic subtask-selection policy, while $\tilde{\pi}_c : \mathcal{I} \to \mathcal{C}$ is a deterministic capacity-allocation policy.

This paper formalizes the approach as a hierarchical policy optimization problem in which the high-level agent must allocate communication capacity to subtasks to minimize total bandwidth usage subject to a probabilistic task-completion constraint. This leads to the following optimization problem:

TODO: you don't define what these policies are anywhere. Do that here before introducing your problem

$$\min_{\pi, \tilde{\pi}} \quad J_c(\tilde{\pi}) \tag{2a}$$

$$\text{s.t.} \quad \Pr\left(\diamond_{(0,\infty)}(\tilde{s} = \tilde{s}_g)\right) \geq p_g \tag{2b}$$

Let $(\pi^*, \tilde{\pi}^*)$ denote an optimal solution to Equation (2).

where $\diamond_{(0,T)}\phi$ denotes the metric temporal logic (MTL) reachability operator meaning "ϕ becomes true within time horizon T" [19]. Because the objective (Equation (2)) is independent of the low-level policy while the chance constraint (Equation (2b)) depends on subtask success rates, this paper formalizes an extended Dec-POMDP (i.e., a low-level Dec-POMDP coupled with a high-level MDP) that decomposes the task into connected subtasks. This paper solves the resulting problem by optimizing the high-level policy via a mixed-integer bilinear program and obtaining low-level policies with MARL.

A. Extended Dec-POMDP System Model

This work defines the system model as an extended Dec-POMDP in which the low-level environment follows a Dec-POMDP and the high-level environment follows an MDP. Formally, this paper defines the extended Dec-POMDP with the tuple,

$$\mathcal{M}' = (\mathcal{N}', \mathcal{S}', \mathcal{A}', P', \mathcal{O}, O, R, b_0, T, \gamma) \tag{3}$$

where $\mathcal{N}' = \{0\} \cup \mathcal{N}$ is the set of agents, where agent 0 is the high-level agent and $\mathcal{N} = \{1, \ldots, N\}$ is the set of low-level agents, $\mathcal{S}' = \mathcal{S} \times \tilde{\mathcal{S}}$ is the extended, factored state space where $\mathcal{S}$ is the low-level state space and $\tilde{\mathcal{S}}$ is the high-level state space, $\mathcal{A}' = \mathcal{A} \times \tilde{\mathcal{A}}_e \times \tilde{\mathcal{A}}_c$ is the action set where $\mathcal{A}$ is the set of low-level environment actions, $\tilde{\mathcal{A}}_e$ is the set of high-level environment actions, and $\tilde{\mathcal{A}}_c$ is the set of high-level communication capacity (e.g., bandwidth) allocation actions, P' is the joint high- and low-level dynamics which we define in-detail later in our formulation, $\mathcal{O}'$ is the joint observation set, $O' : \mathcal{S}' \times \mathcal{A}' \to \Delta(\mathcal{O}')$ is the joint observation function, $R : \mathcal{S} \times \mathcal{A} \times \tilde{\mathcal{A}}_e \to \mathbb{R}$ is the reward function for the low-level agents where $\tilde{a}_e \in \tilde{\mathcal{A}}_e$ defines the current subtask in the high-level MDP, $b'_0 \in \Delta(\mathcal{S}')$ is the initial state distribution, $T \in \mathbb{R}_{>0}$ is the time horizon for the low-level agents, and $\gamma \in (0, 1]$ is the discount factor for the low-level agents

First, this work defines an abstract, high-level Markov decision process (MDP) model of the task for allocating communication capacity to subtasks. An MDP provides a formal structure and rich class of methods for modeling tasks as a composition of subtasks with uncertain transitions and verifying system properties. This paper defines an abstract MDP with the tuple $\tilde{\mathcal{M}} = (\tilde{\mathcal{N}}, \tilde{\mathcal{S}}, \tilde{\mathcal{A}}, \tilde{P}, \tilde{b}_0)$, where $\tilde{\mathcal{N}} = \{0\}$ denotes the MDP agent's index, $\tilde{\mathcal{S}}$ denotes a finite set of states, $\tilde{\mathcal{A}}$ denotes the action set, $\tilde{P} : \tilde{\mathcal{S}} \times \tilde{\mathcal{A}} \to \Delta(\tilde{\mathcal{S}})$ denotes the probabilistic transition function, and $\tilde{b}_0 \in \Delta(\tilde{\mathcal{S}})$ denotes the initial state distribution. We assume the action set is factored as $\tilde{\mathcal{A}} = \tilde{\mathcal{A}}_e \times \tilde{\mathcal{A}}_c$ where $i = (\tilde{s}, \tilde{s}') \in \tilde{\mathcal{A}}_e$ is an environment action that specifies the intended next state by selecting a subtask to execute and $c \in \tilde{\mathcal{A}}_c$ allocates a communication capacity (e.g., bandwidth) to a low-level policy that executes the subtask. The model contains a single absorbing goal state $\tilde{s}_g$ and an absorbing failure state $\tilde{s}_x$ that models the possibility of failing a subtask. The agent cannot choose to transition into $\tilde{s}_x$ such that $(\tilde{s}, \tilde{s}_x) \notin \tilde{\mathcal{A}}_e \ \forall \ \tilde{s} \in \tilde{\mathcal{S}}$, but may transition to $\tilde{s}_x$ due to the probabilistic transition function. Figure 1 shows the action set $\tilde{\mathcal{A}}$ as well as the failure state in the running example.

For the formulation in this paper, we assume that the directed graph induced by the non-absorbing environment actions $\tilde{\mathcal{A}}_e$ is acyclic. Thus, a non-absorbing state cannot be revisited after a subtask is completed, and every execution contains finitely many subtasks before reaching either $\tilde{s}_g$ or $\tilde{s}_x$. This assumption ensures that the edge-occupancy variables used below are finite.

This paper defines the MDP's transition function by interfacing the model with a low-level Dec-POMDP. Similar to a reward machine [20], the reward function in the extended Dec-POMDP is a function of the current subtask, $i \in \tilde{\mathcal{A}}_e$. Accordingly, this paper represents the low-level Dec-POMDP with a set of Dec-POMDP models, $\mathcal{D} = \{\mathcal{M}_i \mid i \in \tilde{\mathcal{A}}_e\}$, where each $\mathcal{M}_i$ models a subtask with a different reward function $R_i : \mathcal{S} \times \mathcal{A}$ and initial state distribution $b_{0,\tilde{s}}$ where $R_i(s, a) = R'(s, a, \tilde{a}_e = i)$ and $\mathcal{B}_0 = \{b_{0,\tilde{s}} \mid \tilde{s} \in \tilde{\mathcal{S}} \setminus \{\tilde{s}_g, \tilde{s}_x\}\}$ denotes the set of initial state distributions for non-absorbing states. Each Dec-POMDP also has a success condition ϕ_i that specifies conditions for the subtask to be successfully completed. Let $\Pi = \{\pi_{ic} \mid (i, c) \in \tilde{A}\}$ be the set of subpolicies for the Dec-POMDPs where π_{ic} uses communication capacity c. Then, since the success conditions have binary outcomes, a trajectory executed by π_{ic} satisfies ϕ_i as a Bernoulli random variable with probability

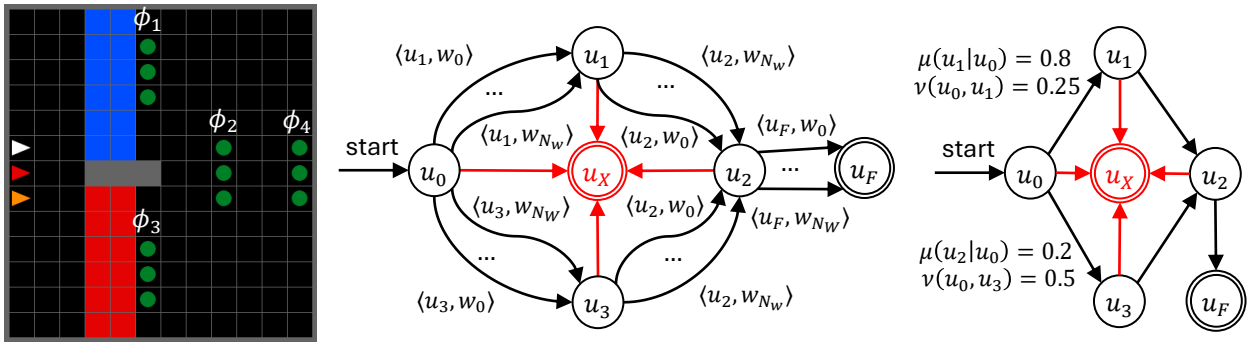


Fig. 1: **Left**: Three agents execute the team navigation task by reaching assigned intermediate goals until they reach the goals on the right side of the map. Each set of goals (green circles) has an associated task specification, ϕ_i. **Middle**: The CM representation of the task before selecting the high-level policies, with subtasks $i = (\tilde{s}, \tilde{s}') \in \mathcal{I}$. **Right**: Selecting $\tilde{\pi}_e$ and $\tilde{\pi}_c$ induces a Markov chain over the CM, where $\tilde{\pi}_e$ selects the next subtask and $\tilde{\pi}_c$ assigns a communication capacity $c \in \mathcal{C}$ to each subtask. Example policy values are provided for $\tilde{s}_0$.

p_{ic}, where $\tilde{\mathcal{P}} = \{p_{ic} \mid (i,c) \in \tilde{\mathcal{A}}\}$ is the set of success rates for all subpolicies. The MDP's dynamics are therefore defined as,

$$\tilde{P}(\alpha \mid \tilde{s}, \tilde{a}_e = i, \tilde{a}_c = c) = \begin{cases} p_{ic} & \alpha = \tilde{s}' \\ 1 - p_{ic} & \alpha = \tilde{s}_x \\ 0 & \text{o.w.} \end{cases}, \quad (4)$$

where Equation (4) defines the relationship between the subpolicies and the high-level reliability constraint (Equation (2b)).

In our case, we focus on reachability success conditions given in MTL as $\phi_i = \diamond_{(0,T)}(s \in \mathcal{S}_{g,i})$ which specifies the team reach some set of joint goal states $\mathcal{S}_{g,i}$ within the time limit T. Figure 1 shows the CM for our running example, where, for example, $s_1 = [(6,2),(6,3),(6,4)]$ is the joint state the team must eventually reach to complete subtask $(\tilde{s}_0, \tilde{s}_1)$.

B. Estimating Subpolicy Success Rates

This paper estimates each success rate p_{ic} by first optimizing a low-level policy π^*_{ic} for Dec-POMDP $\mathcal{M}_i$. In principle, any Comm-MARL algorithm that allows specification of a fixed communication capacity c can be used to learn π^*_{ic}. For example, c could represent the maximum number of edges within a coordination graph when using CASEC [7] or the maximum number of agents that can broadcast messages when using SchedNet [4]. Note that π^*_{ic} is an option policy [21] which is initiated by the MDP agent executing subtask i with communication capacity c and which terminates by satisfying or failing to satisfy ϕ_i. Given the binary success conditions, we estimate the success rate as the maximum likelihood estimator (MLE) of a Bernoulli random variable, $\hat{p}_{ic} = 1/N_p \sum_{k=1}^{N_p} \mathbb{1}_{\{\phi_i\}}\big|_{\pi^*_{ic}}$ where N_p is the number of evaluation episodes and $\mathbb{1}_{\{\phi_i\}}$ is one when ϕ_i is true and zero otherwise.

C. Optimizing the High-Level Agent's Policy

This paper formulates the high-level MDP agent's policy as $\tilde{\pi} = (\tilde{\pi}_e, \tilde{\pi}_c)$, where the subtask-selection policy is stochastic, $\tilde{\pi}_e : \tilde{\mathcal{S}} \to \Delta(\tilde{\mathcal{A}}_e)$, and the communication capacity allocation policy is deterministic, $\tilde{\pi}_c : \tilde{\mathcal{A}}_e \to \tilde{\mathcal{A}}_c$. Given these assumptions, the edge-occupancy variables below represent the expected number of times each subtask is initiated before absorption. Following [22], [23], this paper formulates Equation (2) as a mixed-integer bilinear optimization problem,

$$\min_{x,y} \quad J_c = \frac{1}{Z_c} \sum_{i \in \tilde{\mathcal{A}}_e} x_i \sum_{c \in \tilde{\mathcal{A}}_c} y_{ic} c \quad (5a)$$

s.t.

$$\sum_{c \in \tilde{\mathcal{A}}_c} y_{ic} = 1 \qquad i \in \tilde{\mathcal{A}}_e \quad (5b)$$

$$\sum_{k \in \text{succ}(\tilde{s})} x_k = \mathbb{1}_{\{\tilde{s} = \tilde{s}_0\}} + \sum_{j \in \text{pred}(\tilde{s})} x_j \sum_{c \in \tilde{\mathcal{A}}_c} y_{jc} p_{jc} \forall\ \tilde{s} \in \tilde{\mathcal{S}} \setminus \{\tilde{s}_g\} \quad (5c)$$

$$\sum_{j \in \text{pred}(\tilde{s}_g)} x_j \sum_{c \in \tilde{\mathcal{A}}_c} y_{jc} p_{jc} \geq p_g \quad (5d)$$

$$x_i \in \mathbb{R}_{\geq 0} \qquad i \in \tilde{\mathcal{A}}_e \quad (5e)$$

$$y_{ic} \in \{0,1\} \qquad (i,c) \in \tilde{\mathcal{A}}. \quad (5f)$$

Here, Equation (5a) defines the problem as minimizing the communication cost, measured as a weighted sum of communication capacities across all subtasks, where the normalization constant Z_c is the number of subtasks in the longest path between the initial state $\tilde{s}_0$ and goal state $\tilde{s}_g$. Equation (5e) defines x_i as the decision variable that chooses the expected number of times subtask i is completed by an agent that starts in initial state $\tilde{s}_0$ and uses a stochastic

subtask selection policy given as,

$$\tilde{\pi}_e^*(i \mid \tilde{s}) = \begin{cases} \frac{x_i}{Z_e} & Z_e > 0, i = (\tilde{s}, \cdot) \\ \frac{1}{|\text{succ}(\tilde{s})|} & \text{o.w.} \end{cases} \quad (6)$$

where $Z_e = \sum_{j \in \text{succ}(\tilde{s})} x_j$ normalizes to form a probability distribution and $\text{succ}(\tilde{s}) = \{i \mid i = (\tilde{s}, \cdot),\ i \in \tilde{\mathcal{A}}_e\}$ is the set of successor subtasks leaving $\tilde{s}$. Equations (5b) and (5f) define an indicator function $y_i : \tilde{\mathcal{A}}_c \to \{0, 1\}$ which takes a value of 1 at the single communication capacity allocated to subtask i and induces a deterministic communication capacity policy,

$$\tilde{\pi}_c^*(i) = \sum_{c \in \tilde{\mathcal{A}}_c} y_{ic} c, \quad (7)$$

where $y_{ic} = y_i(c)$. Equation (5c) is a Bellman flow constraint which models the conservation of state-action visitation, x_i, in the MDP. The source indicator $\mathbb{1}_{\{\tilde{s}=\tilde{s}_0\}}$ equals one at the initial state and zero elsewhere, and $\text{pred}(\tilde{s}) = \{i \mid i = (\cdot, \tilde{s}), i \in \tilde{\mathcal{A}}_e\}$ is the set of predecessor subtasks entering $\tilde{s}$. Finally, Equation (5d) enforces the agent's satisfaction of the reliability constraint.

D. Optimizing the Hierarchical Policy

Optimization algorithms are now presented for solving Equation (5) by estimating the global success rate of the hierarchical policy under different simplifying assumptions. First note that each subtask has a reward function, success condition, and set of subpolicies that induce a set of final state distributions (i.e., one for each subpolicy). First, this paper defines conditions that enable different subtasks to interface with one another by assuming that an aggregation function combines the final state distributions for subpolicies *incoming* to $\tilde{s} \in \tilde{\mathcal{S}}$ into the initial state distribution for subpolicies *outgoing* from $\tilde{s}$. Formally, this paper defines the initial state distribution for Dec-POMDPs outgoing from $\tilde{s}$ as,

$$b_{0,\tilde{s}} = f_b(\{b_{f,ic} \mid \forall\ (i,c) \in \tilde{A}_{\tilde{s}}^-\})\ \forall\ \tilde{s} \in \tilde{\mathcal{S}} \setminus \{\tilde{s}_0, \tilde{s}_X\}, \quad (8)$$

where $\tilde{\mathcal{A}}_{\tilde{s}}^- = \{(i,c) \mid (i,c) \in \tilde{\mathcal{A}}, i = (\cdot, \tilde{s})\}$ is set of high-level actions incoming to $\tilde{s}$ and f_b is an aggregation function. Given this dependence, this paper designs two algorithms to solve Equation (5).

1) Independent Subtasks: Specifying final state distributions prior to training subpolicies is typically infeasible since the distributions associated with the optimal policies of less-constrained tasks are typically not known prior to training. Therefore, in this case, we define a constrained version of the success condition ϕ_i such that $\mathcal{S}_{g,i} = \{s_{g,i}\}$ (i.e., each agent must reach an assigned position). Here, specifying $\{\phi_i \mid i \in \tilde{\mathcal{A}}_e\}$ and $b_{0,\tilde{s}_0}$ provides sufficient information to fully specify the set of initial state distributions, $\mathcal{B}_0$, prior to training and satisfy Equation (8). We refer to this class of subtasks as "independent" because pre-specifying $\mathcal{B}_0$ makes the subpolicies independent of subpolicies from prior subtasks and the high-level policy.

Algorithm 1 Optimizing High-Level Policies in a CM with Independent Subtasks

1: **Input**: $\mathcal{C}, p_g, p_{\text{thres}}, N_{\text{tr}}, N_{\text{tr, max}}, N_p$
2: **Output**: $\tilde{\pi}_e^*, \tilde{\pi}_c^*, \Pi^*$
3: $\Pi = \text{InitPolicies}(\mathcal{C})$
4: $\Pi^*, \mathcal{P}^* = \Pi, [0]_{(i,c) \in \mathcal{I} \times \mathcal{C}}$
5: **for** $(i,c) \in \mathcal{I} \times \mathcal{C}$ **do in parallel**
6: **while** $t_{ic} < N_{\text{tr, max}}$ **do**
7: $\pi_{ic}, t_{ic} \leftarrow \text{TrainPolicy}(\pi_{ic}, N_{\text{tr}}, \mathcal{B}_{0,i}), t_{ic} + N_{\text{tr}}$
8: $\hat{p}_{ic} = \text{Evaluate}(\pi_{ic}, N_p)$
9: **if** $(\hat{p}_{ic} > \mathcal{P}_{ic}^*)$ and $(\hat{p}_{ic} > p_{\text{thres}})$ **then**
10: $\Pi_{ic}^*, \mathcal{P}_{ic}^* \leftarrow \pi_{ic}, \hat{p}_{ic}$
11: $\tilde{\pi}_e^*, \tilde{\pi}_c^* = \text{CMOpt}(\mathcal{C}, \mathcal{P}^*, p_g)$ ▷ Solve Equation (5)
12: **return** $\tilde{\pi}_e^*, \tilde{\pi}_c^*, \Pi^*$

Algorithm 1 shows the algorithm for optimizing the hierarchical policy given independent subtasks. Lines 3 and 4 initialize the set of low-level policies, Π^*, and their subtask success rates, $\mathcal{P}^*$. Lines 5 to 10 then define the low-level policy training and evaluation loop, where the algorithm trains each policy for N_{tr} steps, evaluates it for N_p episodes to estimate its success rate, and saves the current-best policy. Finally, Line 11 optimizes the high-level policies.

2) Dependent Subtasks: In the second case, this paper allows less-constrained ϕ_i, where $\mathcal{S}_{g,i}$ is a set of states (i.e., each agent must reach *any* of the positions in $\mathcal{S}_{g,i}$ to complete the task). Here we assume that the paper specifies $\mathcal{B}_0 = \{b_{0,\tilde{s}_0}\}$ before training and estimates each initially unknown $b_{0,\tilde{s}}$ for $\tilde{s} \in \tilde{\mathcal{S}} \setminus \{\tilde{s}_0, \tilde{s}_g, \tilde{s}_x\}$ during training. This approach is related to work in hierarchical reinforcement learning (HRL) in which subgoals are discovered during the training process [24]. In this approach, this paper pre-specifies the success condition ϕ_i (typically denoted in HRL as β_i) but estimates the low-level state distributions during training.

Algorithm 2 describes the approach to training a hierarchical policy in a setting where the algorithm learns the set of initial state distributions, $\mathcal{B}_0$, during each iteration j by performing a forward pass through the CM and sequentially training low-level agents to solve CM subtasks starting from u_I and stepping through the CM until the algorithm completes all paths leading to u_F. In the 0th iteration, the algorithm initializes μ^0 and ν^0 as uniform policies over their available actions at each state, then uses the success probability data from the low-level policies trained in iteration i to solve for μ^1, ν^1. Thus, the algorithm generates the high-level policies in iteration i from data produced by low-level policies trained in iteration $i-1$, then uses those high-level policies to train low-level policies in iteration i.

The following equations describe the estimation of $b_{0,\tilde{s}}$. Based on [25, Equation 4.6], this paper estimates $b_{0,\tilde{s}}$ as a stratified sample across incoming subpolicies, which allows Equation (8) to be rewritten as,

$$\hat{b}_{0,\tilde{s}}^{(j)} = \frac{1}{Z_b} \sum_{(i,c) \in \tilde{\mathcal{A}}_{\tilde{s}}^-} \tilde{\pi}_e^{(j)}(\tilde{s} \mid i) \tilde{\pi}_c^{(j)}(i) \hat{b}_{f,ic}^{(j)} \quad (9)$$

where j is the training iteration and $Z_b = \sum_{(i,c)\in\tilde{\mathcal{A}}_{\tilde{s}}^-} \tilde{\pi}_e^{(j)}(\tilde{s} \mid i)\tilde{\pi}_c^{(j)}(i)$. Because this paper assumes no prior knowledge of each subpolicy's final state distribution, $b_{f,ic}$, it uses the following worst-case number of samples,

$$N_b = \max_m \frac{z^2(1-1/m)}{md^2}, \tag{10}$$

where z is the upper $\alpha/(2m)$ point of the standard normal distribution and m is an integer [26]. Sampling this number of final states ensures that the estimated probability for each state is within d of the true probability with confidence α. Due to the high-level MDP's transition structure, we then estimate $b_{f,ic}$ as the MLE based on N_b successful episodes,

$$\hat{b}_{f,ic}(s) = \frac{1}{N_b}\sum_{k=1}^{N_b} \mathbb{1}((s_{f,k} = s) \wedge (\phi_{i,k}))\Big|_{\pi=\pi_{e,t_{\max}}}, \tag{11}$$

where $s_{f,k}$ is the final state for episode k. Note that, since we are estimating the *successful* final state distribution, N_b is the number of successfully completed episodes to be sampled, and not the total number of episodes to be sampled. Each edge is also assumed to train multiple replicate policies, so

$$\hat{b}_{f,ic} = \frac{1}{M}\sum_{k=1}^{M} \hat{b}_{f,ic,k}, \tag{12}$$

where M is the number of replicates that achieve $\hat{p}_e \geq p_{\text{thres}}$ to avoid sampling final state distributions for policies that perform poorly.

After training the hierarchical policy, we deploy it using the control loop shown in Figure 2. We design the interface between high- and low-level agents to use sparse communication such that the agents only communicate at the start and end of each subtask. This means the high-level agent does not need to maintain an internal representation of the low-level state, which may require significant time and bandwidth to communicate.

Our approach to optimizing communication can accommodate different types of Comm-MARL algorithms. Let $C = |\tilde{\mathcal{A}}_c|$ be the number of bandwidth allocations we consider in solving Equations (2) to (2b). Some Comm-MARL methods like CASEC [7] require specifying the bandwidth allocation prior to training (Type 1), while others like MAIC [27] allow changing bandwidth after training (Type 2). To study performance at C different bandwidths, Type 1 algorithms require training C low-level policies, while Type 2 require training only one policy. We formulate our framework assuming Type 1 algorithms since a similar approach applies to Type 2 with minor modifications.

Rather than train a single policy to solve all subtasks, which may be computationally infeasible given exploration challenges in MARL, we decompose the global problem by training one set of policies per subtask. This requires training $C|\tilde{\mathcal{A}}_e|$ policies for Type 1 algorithms and $|\tilde{\mathcal{A}}_e|$ policies for Type 2 algorithms.

Algorithm 2 Optimizing Coordination Machines with Dependent Subtasks (notation consistent with text)

Input: $\mathcal{C}$, $b_{0,\tilde{s}_0}$, p_g, p_{thres}, N_{tr}, $N_{\text{tr,max}}$, $j_{\max}$, N_p, N_b
Output: $\tilde{\pi}_e^*$, $\tilde{\pi}_c^*$, Π^*, $\hat{\tilde{\mathcal{P}}}^*$, J^*
$j \leftarrow 0$
$\hat{\tilde{\mathcal{P}}}^* \leftarrow 0$, $J^* \leftarrow \infty$, $\Pi \leftarrow \text{InitPolicies}(\mathcal{C})$
while $j \leq j_{\max}$ **do**
  if $j = 0$ **then**
    $\tilde{\pi}_e^{(j)}, \tilde{\pi}_c^{(j)} \leftarrow \text{UniformPolicies}(\mathcal{C})$
  else
    $\tilde{\pi}_e^{(j)}, \tilde{\pi}_c^{(j)} \leftarrow \text{CMOpt}\big(\mathcal{C},\ \tilde{\mathcal{P}}_{t_{\max}}^{(j-1)},\ p_g\big)$ ▷ solve Equation (5a)–Equation (5d)
  $\tilde{\mathcal{P}}_{t_{\max}}^{(j)}, \Pi^{(j)} \leftarrow \text{CMPass}\big(\Pi,\ b_{0,\tilde{s}_0},\ \tilde{\pi}_e^{(j)},\ \tilde{\pi}_c^{(j)},\ N_{\text{tr}},\ N_{\text{tr,max}},\ p_{\text{thres}}\big)$ ▷ train/evaluate low-level policies; returns estimated success rates and trained policies
  if $j \geq 1$ **then**
    $J^{(j)} \leftarrow \text{GetCost}\big(\tilde{\pi}_e^{(j)}, \tilde{\pi}_c^{(j)}, \tilde{\mathcal{P}}_{t_{\max}}^{(j)}\big)$ ▷ evaluate objective Equation (5a)
    $\hat{p}^{(j)} \leftarrow \text{SuccessProb}\big(\tilde{\pi}_e^{(j)}, \tilde{\pi}_c^{(j)}, \tilde{\mathcal{P}}_{t_{\max}}^{(j)}\big)$ ▷ evaluate constraint Equation (5d)
    if $(\hat{p}^{(j)} \geq p_g) \wedge (J^{(j)} < J^*)$ **then**
      $\tilde{\pi}_e^*, \tilde{\pi}_c^*, \Pi^*, \hat{\tilde{\mathcal{P}}}^*, J^* \leftarrow \tilde{\pi}_e^{(j)}, \tilde{\pi}_c^{(j)}, \Pi^{(j)}, \tilde{\mathcal{P}}_{t_{\max}}^{(j)}, J^{(j)}$
  $j \leftarrow j + 1$
return $\tilde{\pi}_e^*$, $\tilde{\pi}_c^*$, Π^*, $\hat{\tilde{\mathcal{P}}}^*$, J^*

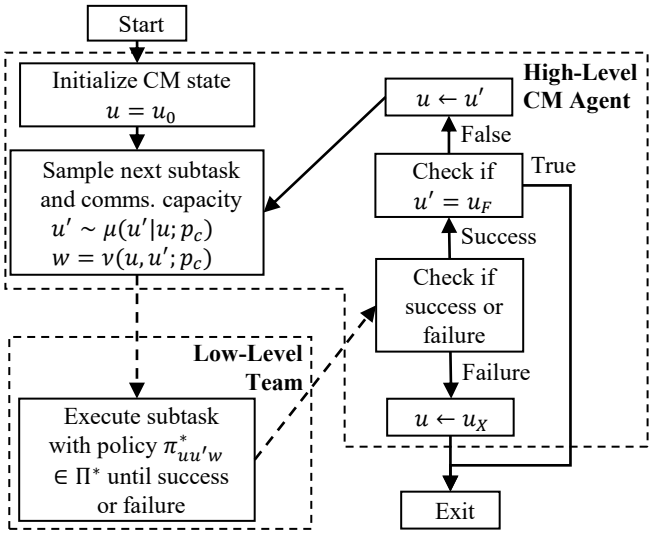


Fig. 2: The control loop for executing the optimized policies in our hierarchical system. The dashed arrows indicate communication between the high- and low-level agents.

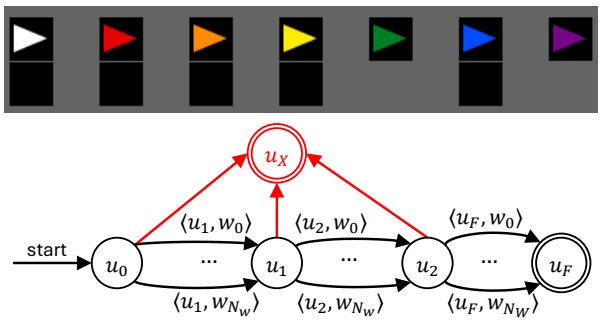


Fig. 3: **Top**: Visualization of our one-step cooperative environment with $N_1 = 5$. **Bottom**: The CM we use to model subtasks in this environment, where each subtask has a different N_{thres}.

V. Experiments and Discussion

A. Environments and Training Details

This paper uses the one-step cooperative environment shown in Figure 3 (top) to demonstrate the method for different-sized teams. Agents are randomly assigned to be either Type 0, in which case they can only take a "still" action, or Type 1, in which case they additionally have a "down" action. At the start of each episode, the number of Type 1 agents, N_1 is uniformly sampled from $\{1, 2, \ldots, N\}$. The objective is for all Type 1 agents to take the "down" action if $N_1 \geq N_{\text{thres}}$, and to take the "still" action otherwise. Agents only observe their own type and the threshold N_{thres}. This paper defines the subtask reward functions as,

$$R(s,a) = \begin{cases} 1 & \phi_0 \\ -\dfrac{N_{\text{down}}}{N_1} & \phi_1, \end{cases} \tag{13}$$

where N_{down} is the number of agents that take the "down" action, $\phi_0 = \big((N_1 \geq N_{\text{thres}}) \wedge (N_{\text{down}} = N_1)\big) \vee \big((N_1 < N_{\text{thres}}) \wedge (N_{\text{down}} = 0)\big)$ is the condition for success, and $\phi_1 = \neg\phi_0$ is the condition for failure.

The team navigation environment shown in Figure 1 is also used to demonstrate the method on a more-realistic task. Here, agents can move in the four cardinal directions or stay still. Each agent has a 0.5% probability (per time step) of being visually detected in the blue and red zones, and a 10% probability (per time step) of being detected via radio communication in the red zone when using a non-zero level of communication. This paper defines the subtask reward functions as $r_t = 0.9N_{\text{reach},t} - N_{\text{leave},t} + I_{\text{all},t}$, where $N_{\text{reach},t}$ counts the agents that reach their goal at time t, $N_{\text{leave},t}$ counts the agents that leave their goal at time t, and $I_{\text{all},t} = 1$ if all agents are at their goal states at time t and 0 otherwise. Episodes terminate in a success if all agents reach their assigned goal, and terminate in a failure if any agent is detected or 100 time steps pass.

In both environments, this paper uses Algorithm 1 to train the hierarchical agent. The algorithm trains each low-level policy $\pi^*_{uu'w}$ using CASEC [7] and sets the communication capacity to $w = \lambda \in [0, 1]$, where λ controls the number of directed edges in the coordination graph. In the one-step environment, this paper trains each policy for a total of $N_{\text{tr, max}} = 25,000$ steps. Each CM training iteration trains the policies for $N_{\text{tr}} = 1,000$ steps and then evaluates them for $N_p = 100$ episodes to estimate the success rate. In the team navigation environment, this paper trains the policies for a total of $N_{\text{tr, max}} = 1.5$ million steps, with $N_{\text{tr}} = 25,000$ training steps and $N_p = 100$ evaluation episodes. This paper trains the policies with three random seeds while holding all other hyperparameters constant, and uses Gurobi 11.0 [28] to solve Equation (5).

B. Results from One-Step Cooperative Environment

The method is compared to a non-hierarchical baseline by solving Equations (5a) to (5d) with an additional constraint that fixes the communication capacity across all subtasks. This baseline approximates the fixed-communication capacity approach taken by many existing Comm-MARL methods. Figure 4 (left) shows the estimated success rate for each communication capacity and subtask for $N = 7$ agents. We see that increasing communication capacity increases success rate (up to a certain point), though we note that this is not always the case. For example, for $N = 9$ (not shown), we see a non-monotonic relationship between communication capacity and success rate, likely due to learning instabilities of CASEC or its use of the Max-Sum algorithm for action selection. Figure 4 (middle) shows the optimized communication cost for various p_g requirements. The method incurs a communication cost less than or equal to the baseline at all levels of p_g. Figure 4 (right) shows the overall cost difference between our method and the baseline as function of the number of agents N, where the aggregated percentage decease in cost is calculated as,

$$\frac{\sum_{p_g \in P_g} J^*_{\text{baseline},p_g} - J^*_{\text{ours},p_g}}{\sum_{p_g \in P_g} J^*_{\text{baseline},p_g}}, \tag{14}$$

where $P_g = \{0, 0.1, ..., 1\}$. The method produces cost savings across all team sizes, with savings of up to 81%. These cost savings do not monotonically increase as the number of agents increases, potentially due to scaling issues with CASEC leading to generally lower subtask success rates for large teams.

C. Results from Team Navigation Environment

Figure 5 shows optimized communication costs for various p_g requirements for our team navigation environment. We again see that our method incurs a cost less than or equal to the baseline at all sampled values of p_g, with a 33% median reduction in cost across the sampled values of p_g and a maximum cost reduction of 50% for $p_g = 0.91$. We do not show results for our baseline for $p_g > 0.95$, since no feasible solutions existed for the baseline in those cases due to some subtasks having non-monotonic success rate trends, where increasing communication capacity decreased performance. These results support the key thesis of this work, namely, that using a hierarchical task model allows us to ensure an MAS

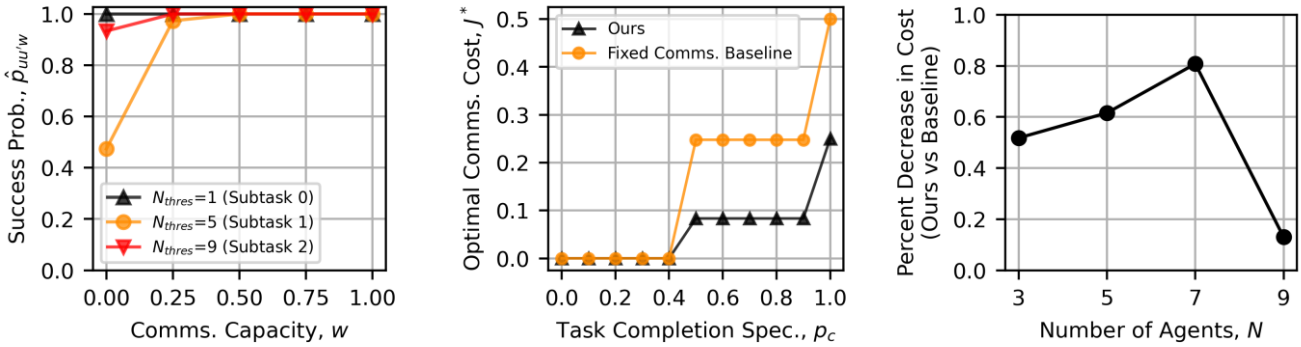


Fig. 4: **Left**: Success rates from our one-step cooperative environment with $N = 7$. **Middle**: Optimized communication cost for various p_g requirements with $N = 7$. Our method incurs a cost less than or equal to our baseline for all sampled values of p_g. **Right** The overall communication cost reduction incurred by our method across all sampled values of p_g for various team sizes. Our method reduces overall communication cost for all considered team sizes, with a maximum reduction of 81% for $N = 7$.

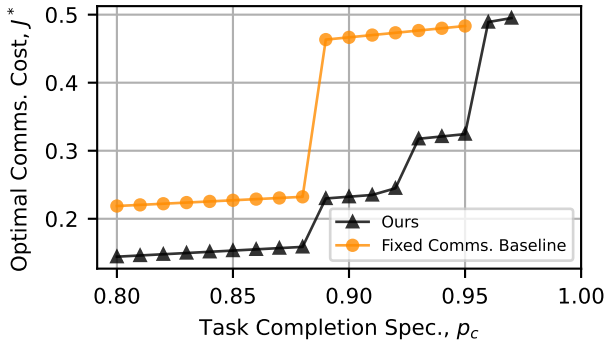


Fig. 5: Optimized communication cost for various p_g requirements. Our method incurs a cost less than or equal to our baseline for all sampled values of p_g. The baseline is unable to satisfy the task specification for $p_g > 0.95$.

satisfies a given level of performance while minimizing inter-agent communication, while at the same time ensuring the global communication cost is less than or equal to a baseline fixed-communication model.

Finally, we find the gradient of the optimal cost function, J^*, is nonlinear. In particular, the communication cost increases by 30% when p_g increases from 0.92 to 0.93 and 51% when p_g increases from 0.95 to 0.96. These are significant compared to the increase in cost of only 1% when p_g is increased from 0.93 to 0.94. This analysis is useful because it allows us to identify regions where small changes in p_g can cause large changes in the communication cost, which could inform design trade-offs between system performance and the communication network.

VI. Conclusions

This work develops a novel hierarchical method for designing an MAS that satisfies a probabilistic task-completion specification while minimizing communication cost. Our method is based on a CM model that decomposes a task into a PFA defined over subtasks, which we use to formulate an optimization problem that solves for high-level policies to choose subtasks to execute and allocate communication capacities to each subtask. We show that our method provides a significant reduction in communication cost compared to a non-hierarchical baseline in simulated environments, and demonstrate its use in a multi-robot system. Future work includes integration of more-realistic communication models and ways to address partial satisfaction of subtask specifications.

References

[1] A. Smit, H. A. Engelbrecht, W. Brink, and A. Pretorius, "Scaling multi-agent reinforcement learning to full 11 versus 11 simulated robotic football," *Auton Agent Multi-Agent Syst*, vol. 37, no. 1, p. 20, 2023. [Online]. Available: https://link.springer.com/10.1007/s10458-023-09603-y

[2] T. Chu, J. Wang, L. Codecà, and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE transactions on intelligent transportation systems*, vol. 21, no. 3, pp. 1086–1095, 2019.

[3] C. Zhu, M. Dastani, and S. Wang, "A survey of multi-agent deep reinforcement learning with communication," *Autonomous Agents and Multi-Agent Systems*, vol. 38, no. 1, p. 4, 2024.

[4] D. Kim, S. Moon, D. Hostallero, W. J. Kang, T. Lee, K. Son, and Y. Yi, "Learning to Schedule Communication in Multi-agent Reinforcement Learning," in *ICLR 2019*, 2019.

[5] R. Wang, X. He, R. Yu, W. Qiu, B. An, and Z. Rabinovich, "Learning efficient multi-Agent communication: An information bottleneck approach," in *37th International Conference on Machine Learning, ICML 2020*, 2020, pp. 9850–9860.

[6] W. Böhmer, V. Kurin, and S. Whiteson, "Deep coordination graphs," in *International Conference on Machine Learning*. PMLR, 2020, pp. 980–991.

[7] T. Wang, L. Zeng, W. Dong, Q. Yang, Y. Yu, and C. Zhang, "Context-aware sparse deep coordination graphs," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=wQfgfb8VKTn

[8] A. Singh, T. Jain, and S. Sukhbaatar, "Learning when to communicate at scale in multiagent cooperative and competitive tasks," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=rye7knCqK7

[9] A. Das, T. Gervet, J. Romoff, D. Batra, D. Parikh, M. Rabbat, and J. Pineau, "TarMAC: Targeted Multi-Agent Communication," in *Proceedings of the 36th International Conference on Machine Learning*, 2019.

[10] H. Mao, Z. Zhang, Z. Xiao, Z. Gong, and Y. Ni, "Learning Agent Communication under Limited Bandwidth by Message Pruning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 2020, pp. 5142–5149. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/5957

[11] M. Ghasemi, A. Nazari, and D. Thiruvady, "Review of recent advances on the resource-constrained project scheduling problem," *International Journal of Production Research*, pp. 1–44, 2026.

[12] Y. Niu, R. Paleja, and M. Gombolay, "Magic: Multi-agent graph-attention communication," in *Mair2 Workshop at International Conference on Computer Vision (ICCV)*, 2021.

[13] S. Li, J. K. Gupta, P. Morales, R. Allen, and M. J. Kochenderfer, "Deep implicit coordination graphs for multi-agent reinforcement learning," in *AAMAS*, 2021, pp. 764–772.

[14] L. Yuan, Z. Zhang, L. Li, C. Guan, and Y. Yu, "A survey of progress on cooperative multi-agent reinforcement learning in open environment," *arXiv preprint arXiv:2312.01058*, 2023.

[15] T. Phan, F. Ritz, L. Belzner, P. Altmann, T. Gabor, and C. Linnhoff-Popien, "VAST: Value Function Factorization with Variable Agent Sub-Teams," in *35th Conference on Neural Information Processing Systems (NeurIPS 2021)*, 2021.

[16] D. Radke, K. Larson, T. Brecht, and K. Tilbury, "Towards a Better Understanding of Learning with Multiagent Teams," in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. Macau, SAR China: International Joint Conferences on Artificial Intelligence Organization, 2023, pp. 271–279. [Online]. Available: https://www.ijcai.org/proceedings/2023/31

[17] S. Smith, C. Neary, and U. Topcu, "Automatic Decomposition of Reward Machines for Decentralized Multiagent Reinforcement Learning," in *2023 62nd IEEE Conference on Decision and Control (CDC)*. Singapore, Singapore: IEEE, 2023, pp. 5423–5430. [Online]. Available: https://ieeexplore.ieee.org/document/10384224/

[18] W. Duan, J. Lu, and J. Xuan, "Group-Aware Coordination Graph for Multi-Agent Reinforcement Learning," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. Jeju, South Korea: International Joint Conferences on Artificial Intelligence Organization, 2024, pp. 3926–3934. [Online]. Available: https://www.ijcai.org/proceedings/2024/434

[19] J. Ouaknine and J. Worrell, "Some recent results in metric temporal logic," in *International Conference on Formal Modeling and Analysis of Timed Systems*. Springer, 2008, pp. 1–13.

[20] R. T. Icarte, T. Klassen, R. Valenzano, and S. McIlraith, "Using reward machines for high-level task specification and decomposition in reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2018, pp. 2107–2116.

[21] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in RL," *Artificial Intelligence*, vol. 112, pp. 181–211, 1999.

[22] M. L. Puterman, *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

[23] K. Etessami, M. Kwiatkowska, M. Y. Vardi, and M. Yannakakis, "Multi-objective model checking of markov decision processes," *Logical Methods in Computer Science*, vol. 4, 2008.

[24] S. Pateria, B. Subagdja, A.-h. Tan, and C. Quek, "Hierarchical reinforcement learning: A comprehensive survey," *ACM Computing Surveys (CSUR)*, vol. 54, no. 5, pp. 1–35, 2021.

[25] S. L. Lohr, *Sampling: design and analysis*. Chapman and Hall/CRC, 2021.

[26] S. K. Thompson, "Sample size for estimating multinomial proportions," *The American Statistician*, vol. 41, no. 1, pp. 42–46, 1987.

[27] L. Yuan, J. Wang, F. Zhang, C. Wang, Z. Zhang, Y. Yu, and C. Zhang, "Multi-agent incentive communication via decentralized teammate modeling," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 36, no. 9, 2022, pp. 9466–9474.

[28] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2024. [Online]. Available: https://www.gurobi.com